

```
# Basic libraries
import re
import math
from collections import Counter, defaultdict

# NLP library
import nltk
nltk.download('punkt')
nltk.download('stopwords')

from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

```
text = """
Once upon a time, there was a small village near a river.
People talked, laughed, and shared stories every evening.
"How was your day?" asked Maria.
"It was long but good," replied John.
They believed language connects hearts and minds.
""" * 60 # repeats text to reach ~1200 words
```

```
print("Sample text:\n")
print(text[:500])
```

Sample text:

Once upon a time, there was a small village near a river.
People talked, laughed, and shared stories every evening.
"How was your day?" asked Maria.
"It was long but good," replied John.
They believed language connects hearts and minds.

Once upon a time, there was a small village near a river.
People talked, laughed, and shared stories every evening.
"How was your day?" asked Maria.
"It was long but good," replied John.
They believed language connects hearts and minds.

Once upon a time, there

```
def preprocess_text(text):
    # lowercase
    text = text.lower()

    # remove punctuation & numbers
    text = re.sub(r'^a-z\s|', '', text)

    # sentence tokenization
    sentences = sent_tokenize(text)

    processed_sentences = []

    for sent in sentences:
        words = word_tokenize(sent)
        # add start & end tokens
```

```
processed_sentences.append(['<s>'] + words + ['</s>'])

return processed_sentences
```

```
nltk.download('punkt_tab')
sentences = preprocess_text(text)
print(sentences[0])
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
['<s>', 'once', 'upon', 'a', 'time', 'there', 'was', 'a', 'small', 'village', 'near', 'a', 'river', 'people
```

```
unigrams = Counter()
for sent in sentences:
    unigrams.update(sent)
```

```
bigrams = Counter()
for sent in sentences:
    for i in range(len(sent)-1):
        bigrams[(sent[i], sent[i+1])] += 1
```

```
trigrams = Counter()
for sent in sentences:
    for i in range(len(sent)-2):
        trigrams[(sent[i], sent[i+1], sent[i+2])] += 1
```

```
print("Top 10 Unigrams:", unigrams.most_common(10))
print("Top 10 Bigrams:", bigrams.most_common(10))
print("Top 10 Trigrams:", trigrams.most_common(10))
```

```
Top 10 Unigrams: [('a', 180), ('was', 180), ('and', 120), ('once', 60), ('upon', 60), ('time', 60), ('there', 60), ('a', 60), ('the', 60), ('in', 60)]
Top 10 Bigrams: [ (('once', 'upon'), 60), (('upon', 'a'), 60), (('a', 'time'), 60), (('time', 'there'), 60), (('there', 'was'), 60), (('was', 'a'), 60), (('a', 'small'), 60), (('small', 'village'), 60), (('village', 'near'), 60), (('near', 'a'), 60)]
Top 10 Trigrams: [ (('once', 'upon', 'a'), 60), (('upon', 'a', 'time'), 60), (('a', 'time', 'there'), 60), (('there', 'was', 'a'), 60), (('was', 'a', 'small'), 60), (('a', 'small', 'village'), 60), (('small', 'village', 'near'), 60), (('village', 'near', 'a'), 60), (('near', 'a', 'river'), 60), (('river', 'people', 'the'), 60)]
```

```
vocab_size = len(unigrams)

def unigram_prob(word):
    return (unigrams[word] + 1) / (sum(unigrams.values()) + vocab_size)

def bigram_prob(w1, w2):
    return (bigrams[(w1, w2)] + 1) / (unigrams[w1] + vocab_size)

def trigram_prob(w1, w2, w3):
    return (trigrams[(w1, w2, w3)] + 1) / (bigrams[(w1, w2)] + vocab_size)
```

```
def sentence_probability(sentence, model="unigram"):
    words = ['<s>'] + word_tokenize(sentence.lower()) + ['</s>']
    prob = 1

    if model == "unigram":
        for w in words:
            prob *= unigram_prob(w)

    elif model == "bigram":
        for i in range(len(words)-1):
```

```

        prob *= bigram_prob(words[i], words[i+1])

    elif model == "trigram":
        for i in range(len(words)-2):
            prob *= trigram_prob(words[i], words[i+1], words[i+2])

    return prob

```

```

test_sentences = [
    "people talked and laughed",
    "language connects minds",
    "the village was quiet",
    "john replied quickly",
    "stories were shared"
]

for s in test_sentences:
    print("\nSentence:", s)
    print("Unigram:", sentence_probability(s, "unigram"))
    print("Bigram:", sentence_probability(s, "bigram"))
    print("Trigram:", sentence_probability(s, "trigram"))

```

```

Sentence: people talked and laughed
Unigram: 5.218720823337302e-13
Bigram: 1.1202923830594708e-08
Trigram: 2.0352749788789342e-07

```

```

Sentence: language connects minds
Unigram: 1.0519388502578246e-11
Bigram: 3.5177180828067374e-06
Trigram: 7.530517421852056e-06

```

```

Sentence: the village was quiet
Unigram: 2.0979619115630322e-16
Bigram: 9.132327700523959e-10
Trigram: 5.335720890574504e-07

```

```

Sentence: john replied quickly
Unigram: 1.72448991845545e-13
Bigram: 7.559119495619264e-08
Trigram: 1.9742167295125664e-05

```

```

Sentence: stories were shared
Unigram: 1.7244899184554496e-13
Bigram: 7.559119495619264e-08
Trigram: 1.9742167295125664e-05

```

```

def perplexity(sentence, model="unigram"):
    words = ['<s>'] + word_tokenize(sentence.lower()) + ['</s>']
    N = len(words)
    log_prob = 0

    if model == "unigram":
        for w in words:
            log_prob += math.log(unigram_prob(w))

    elif model == "bigram":
        for i in range(len(words)-1):
            log_prob += math.log(bigram_prob(words[i], words[i+1]))

    elif model == "trigram":
        for i in range(len(words)-2):
            log_prob += math.log(trigram_prob(words[i], words[i+1], words[i+2]))

```

```
return math.exp(-log_prob / N)
```

```
for s in test_sentences:  
    print("\nSentence:", s)  
    print("Unigram PP:", perplexity(s, "unigram"))  
    print("Bigram PP:", perplexity(s, "bigram"))  
    print("Trigram PP:", perplexity(s, "trigram"))
```

```
Sentence: people talked and laughed  
Unigram PP: 111.4480966558352  
Bigram PP: 21.14031412074456  
Trigram PP: 13.038555633645178
```

```
Sentence: language connects minds  
Unigram PP: 156.89239802598104  
Bigram PP: 12.323889446405339  
Trigram PP: 10.583639451022684
```

```
Sentence: the village was quiet  
Unigram PP: 410.2359664706128  
Bigram PP: 32.104783985351645  
Trigram PP: 11.103702468586786
```

```
Sentence: john replied quickly  
Unigram PP: 356.9997138799578  
Bigram PP: 26.564751511423474  
Trigram PP: 8.728126548676135
```

```
Sentence: stories were shared  
Unigram PP: 356.9997138799581  
Bigram PP: 26.564751511423463  
Trigram PP: 8.728126548676135
```

The Bigram model gave lower perplexity than the Unigram model. Trigram performed best when enough context was available. However, trigrams sometimes failed due to data sparsity. Unseen words increased perplexity significantly. Smoothing helped reduce zero probabilities. Bigram models balance context and data availability. Trigram models require large datasets to perform well. Overall, Bigram gave the most stable results.